

day02 课堂笔记

day02 课堂笔记

0. 复习和反馈

1. if 判断语句

- If 判断的基本格式

- if else 结构

- Debug 调试

- if elif 结构

- if 嵌套

- 猜拳游戏

- 三目运算

循环

- 循环的基本语法

- 应用

- 循环嵌套

for 循环遍历

- 循环打印直角三角形

- Break 和 continue

- 循环 else 结构

总结补充

0. 复习和反馈

挺好的

有的地方有点快 听的不打清楚 希望关键重要的地方能提醒下

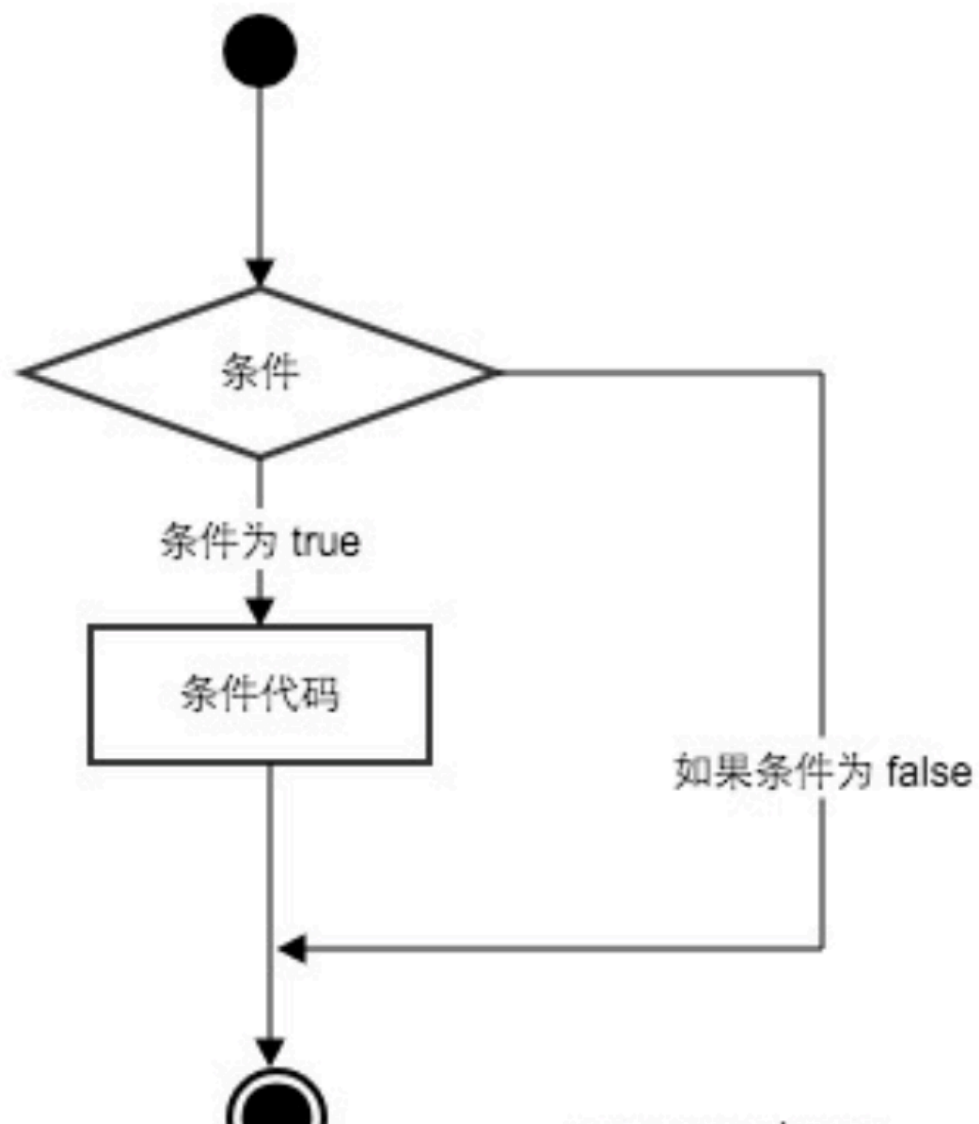
希望老师讲课速度慢一点

前期内容简单一些,老师讲的也很详细,目前暂无问题.

第一天还是有点不清楚,感觉自己回去多练练就没问题了

- 1 单引号和双引号是没有区别的
- 2
- 3 需要将数字类型的字符串转换为数字类型(`int float`),
就可以使用 `eval()`, 也可以不适用, 直接是所有 `int()`
或者 `float()`

1. if 判断语句



If 判断的基本格式

- 1 `if` 判断条件:
- 2 判断条件为 `True`, 会执行的代码
- 3 判断条件为 `True`, 会执行的代码
- 4 ...
- 5
- 6 顶格书写的代码, 代表和 `if` 判断没有关系
- 7 在 `python` 中使用缩进, 代替代码的层级关系, 在 `if` 语句的缩进内, 属于 `if` 语句的代码块 (多行代码的意思)

案例需求：

1. 通过用户键盘输入，获取年龄
2. 判断年龄是否满足18岁，满足输出 哥18岁了，可以进入网吧为所欲为了
3. 程序最后输出， if 判断结束 (不管是否满足，都会输出)

```
1 # 1. 通过用户键盘输入，获取年龄 input()
2 age = input('请输入你的年龄:') # str
3 # 需要将字符串类型的age，转换为 int类型的age
4 age = int(age) # int
5 # 2. 判断年龄是否满足18岁，满足输出 哥18岁了，可以进入网吧为所欲为了
6 if age >= 18: # 字符串和数字不能直接进行比较
7     # 条件满足才会执行
8     print('哥18岁了，可以进入网吧为所欲为了')
9
10 # 3. 程序最后输出，if 判断结束 (不管是否满足，都会输出)
11 print('if 判断结束')
12
```

Run: 02-if 语句

```
if age >= 18

C:\Develop\python38\python.exe "C:/Users/n1/Desktop/43/day02/02-if 语句.py"
请输入你的年龄:20
哥18岁了，可以进入网吧为所欲为了
if 判断结束

Process finished with exit code 0
```

if else 结构

```
1  if 判断条件:
2      判断条件为 True, 会执行的代码
3      判断条件为 True, 会执行的代码
4      ...
5  else:
6      判断条件为 False, 会执行的代码
7      判断条件为 False, 会执行的代码
8      .....
9
10
11  if 和 else 只会执行其中的一个,
```

```
1  # 1. 通过用户键盘输入, 获取年龄 input()
2  age = input('请输入你的年龄:') # str
3  # 需要将字符串类型的age, 转换为 int类型的age
4  age = int(age) # int
5  # 2. 判断年龄是否满足18岁, 满足输出`哥18岁了, 可以进入网吧为所欲为了`
6  if age >= 18:
7      # 条件满足才会执行
8      print('哥18岁了, 可以进入网吧为所欲为了')
9  else:
10     # 判断条件不满足, 会执行的代码
11     print('不满18岁, 回去好好学习吧, 少年!!!')
12
13  # 3. 程序最后输出, `if 判断结束` (不管是否满足, 都会输出)
14  print('if 判断结束')
```

```
Run: 02-if else 结构 x
C:\Develop\python38\python.exe "C:/Users/n1/Desktop/43/day02/02-if else 结构.py"
请输入你的年龄:19
哥18岁了, 可以进入网吧为所欲为了
if 判断结束

Process finished with exit code 0
```

Debug 调试

1. 可以查看代码的执行过程
2. 可以排查错误

步骤:

1. 打断点(一般可以在代码的开始打上断点,或者在查看代码的地方打断点)
2. 右键 debug 运行代码

```
1 # 1. 通过用户键盘输入, 获取年龄 input()
2 age = input('请输入你的年龄:') # str
3 # 需要将字符串类型的age, 转换为 int类型的age
4 age = int(age) # int
5 # 2. 判断年龄是否满足18岁, 满足输出`哥18岁了, 可以进入
6 if age >= 18:
7     # 条件满足才会执行
8     print('哥18岁了, 可以进入网吧')
9 else:
10    # 判断条件不满足, 会
11    print('不满18岁, 回
12
13 # 3. 程序最后输出, `if
14 print('if 判断结束')
```

if age >= 18

Run: 02-if else 结构 ×

C:\Develop\python38\pyt
请输入你的年龄:18
哥18岁了, 可以进入网吧为
if 判断结束

Process finished with exit code 0

会输出

stop/

Context Actions:

- Show Context Actions (Alt+Enter)
- Copy Reference (Ctrl+Alt+Shift+C)
- Paste (Ctrl+V)
- Paste from History... (Ctrl+Shift+V)
- Paste without Formatting (Ctrl+Alt+Shift+V)
- Column Selection Mode (Alt+Shift+Insert)
- Find Usages (Alt+F7)
- Refactor
- Folding
- Go To
- Generate... (Alt+Insert)
- Run '02-if else 结构' (Ctrl+Shift+F10)
- Debug '02-if else 结构'**
- Edit '02-if else 结构'...
- Show in Explorer
- File Path (Ctrl+Alt+F12)
- Open in Terminal
- Local History
- Execute Line in Python Console (Alt+Shift+E)
- Run File in Python Console
- Compare with Clipboard
- Create Gist...

3. 点击 下一步, 查看代码执行的过程

if elif 结构

```
1  if  判断条件1:
2      判断条件1成立, 执行的代码
3  elif 判断条件2:
4      判断条件1不成立, 判断条件2 成立, 会执行的代码
5  else:
6      判断条件1和判断条件2都不成立, 执行的代码
7
8  -----
9  if 判断条件1:
10     判断条件1成立执行的代码
11
12  if 判断条件2:
13     判断条件2 成立执行的代码
```

需求:

1. 成绩大于等于90 , 输出优秀
2. 成绩大于等于80, 小于90, 输出良好
3. 成绩大于等于60, 小于80, 输出及格
4. 小于60, 输出不及格


```

1 score = eval(input('请输入你的成绩:'))
2
3 # 1. 成绩大于等于90，输出优秀
4 ● if score >= 90:           if elif elif else 结构，只有有一个条件满足成立，后续的条件都不会再进行判断了
5     print('优秀')
6 # 2. 成绩大于等于80，小于90，输出良好
7 elif (score >= 80) and score < 90:
8     print('良好')
9 # 3. 成绩大于等于60，小于80，输出及格
10 elif score >= 60: # 想要执行这个判断的前提是，前边两个条件都不满足(成立)，代表score一定小于80
11     print('及格')
12 # 4. 小于60，输出不及格
13 else:
14     print('不及格')
15
16
17 print("程序结束")
18

```

```

1 if 判断条件1:
2     判断条件1成立,执行的代码
3 elif 判断条件2:
4     判断条件1不成立,判断条件2 成立,会执行的代码
5 else:
6     判断条件1和判断条件2都不成立,执行的代码
7

```

if 嵌套

```

1 if 判断条件1:
2     判断条件1 成立,会执行的代码
3     if 判断条件2:
4         判断条件1成立, 判断条件2成立执行的代码
5     else:
6         判断条件1成立, 判断条件2不成立执行的代码
7 else:
8     判断条件1不成立,会执行的代码

```

```

1  # 假设 money 大于等于2 可以上车
2  money = int(input('请输入你拥有的零钱:'))
3  # ctrl x 剪切 ctrl v 粘贴
4
5  # 1. 有钱可以上车
6  if money >= 2:
7      print('我上车了')
8      # 假设 seat 大于等于1, 就可以坐
9      seat = int(input('车上的空位个数:'))
10     # 3. 有空座位, 可以坐
11     if seat >= 1:
12         print('有座位坐')
13     else:
14         # 4. 没有空座位, 就站着
15         print('没有座位, 只能站着')
16 else:
17     # 2. 没钱不能上车, 走路
18     print('没钱, 我只能走路')
19
20

```

if 嵌套，只有外层的条件成立，才会判断内层的 if

```

1  if 判断条件1:
2      判断条件1 成立, 会执行的代码
3      if 判断条件2:
4          判断条件1成立, 判断条件2成立执行的代码
5      else:
6          判断条件1成立, 判断条件2不成立执行的代码
7  else:
8      判断条件1不成立, 会执行的代码

```

猜拳游戏

```

1  import random  # 导入随机数模块
2  # 产生 [a, b] 之间的随机整数, 包含 a 和 b
3  num = random.randint(a, b)

```

```

1  import random
2
3  # 1. 用户输入自己出拳的内容
4  user = int(input('请输入要出的拳:1(石头) 2(剪刀) 3(布):'))
5  # 2. 让电脑随机出拳
6  computer = random.randint(1, 3) # 随机产生1-3 之间的随机数
7  # print(computer)
8  # 3. 判断胜负
9  # 3.1 平局 输入的内容一样 user == computer
10 # 3.2 user 胜利, ①. user==1 and computer==2 ② user==2 and computer==3 ③ user==3 and computer == 1
11 # 3.3 else 剩下的情况就是电脑胜利
12 if user == computer:
13     print('平局')
14 elif (user == 1 and computer == 2) or (user == 2 and computer == 3) or (user == 3 and computer == 1):
15     print('恭喜你, 胜利了')
16 else:
17     print('你输了, 弱爆了')
18
19

```

三目运算

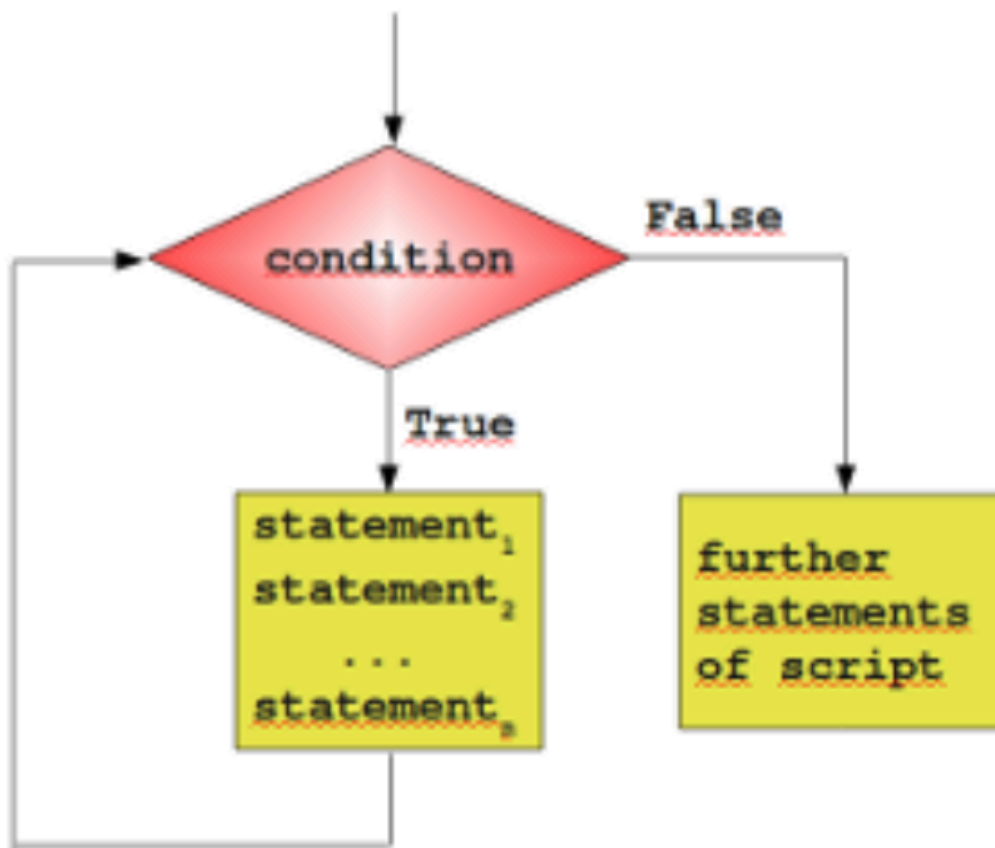
if else 结构变形

```
1  if 判断条件1:
2      表达式1
3  else:
4      表达式2
5
6  判断条件成立, 执行表达式 1, 条件不成立, 执行表达式 2
7
8  变量 = 表达式1 if 判断条件 else 表达式2  # 推荐使用
    扁平化代码
9
10 变量最终存储的结构是:
11      判断条件成立, 表达式1的值,
12      条件不成立, 表达式2的值
```

```
1  a = int(input('请输入一个数字:'))
2  b = int(input('请输入一个数字:'))
3
4  result = a - b if a > b else b - a
5  result = (a - b) if a > b else (b - a)
6  print(result)
```

```
8  变量 = 表达式1 if 判断条件 else 表达式2  # 推荐使用扁平化代码
9
10 变量最终存储的结构是:
11      判断条件成立, 表达式1的值,
12      条件不成立, 表达式2的值
```

循环



循环的基本语法

- 1 `while` 判断条件：
- 2 判断条件成立, 执行的代码
- 3 判断条件成立, 执行的代码
- 4
- 5 不在 `while` 的缩进内, 代表和循环没有关系
- 6
- 7 `while` 和 `if` 的区别：
- 8 `if` 的代码块, 条件成立, 只会执行一次
- 9 `while` 的代码块, 只要条件成立, 就会一直执行

```
1  # 使用print输出模拟跑一圈
2  # print('操场跑圈中.....')
3  # print('操场跑圈中.....')
4  # print('操场跑圈中.....')
5  # print('操场跑圈中.....')
6  # print('操场跑圈中.....')
7
8  # 使用循环解决跑圈问题
9  # 1. 记录已经跑了多少圈
10 i = 0
11 # 2. 书写循环, 判断是否满足条件
12 while i < 5:
13     print('操场跑圈中.....')
14     # 2. 跑了一圈之后, 记录的圈数加1
15     i += 1
16
17 print('跑圈完成')
```

```
1  while True:  # 无限循环
2      代码
3
4
5  死循环: 相当于是代码的 bug, 错误
6  无限循环: 人为书写的, 故意这样写的
```

应用

```

1  # 1 + 2 + 3 + 99 + 100
2  # 循环生成 1- 100 之间的数字
3  # 定义变量记录初始的值
4  my_sum = 0
5  i = 1
6  while i <= 100:
7      # 求和
8      my_sum += i  # my_sum = my_sum + i
9      # 改变i的值
10     i += 1
11

```

Run: 10-计算1-100之间的累加和 ×

C:\Develop\python38\python.exe C:/Users/n1/Desktop/43/day
 求和的结果为 5050
 Process finished with exit code 0

```

1  # 循环生成 1- 100 之间的数字
2  # 定义变量记录初始的值
3  my_sum = 0
4  i = 2 # 1-100 之间第一个偶数
5  while i <= 100:
6      my_sum += i  # my_sum = my_sum + i
7      # 改变i的值
8      i += 2 # 保证每次的结果都是偶数
9
10 # 将代码放在循环的缩进里边. 还是外边, 可以思考, 这行代码想让他打印输出几次, 如果只输出一次, 放在外边,
11 # 如果想要多次输出, 放在里边
12 print('求和的结果为', my_sum)
13

```

```

2  # 循环生成 1- 100 之间的数字
3  # 定义变量记录初始的值
4  my_sum = 0
5  i = 1
6  while i <= 100:
7      # 先判断数字是不是偶数, 如果是偶数求和
8      if i % 2 == 0:
9          my_sum += i  # my_sum = my_sum + i
10     # 改变i的值
11     i += 1
12
13 # 将代码放在循环的缩进里边. 还是外边, 可以思考, 这行代码想让他打印输出几次, 如果只输出一次, 放在外边,
14 # 如果想要多次输出, 放在里边
15 print('求和的结果为', my_sum)
16

```

循环嵌套

```
1 while 判断条件1:
2     代码1
3     while 判断条件2:
4         代码2
5
6 =====
7 代码 1 执行一次,代码会执行多次
```

```
1 # 操场跑圈 一共需要跑5圈
2 # 每跑一圈,需要做3个俯卧撑,
3
4 # 1. 定义变量记录跑的圈数
5 i = 0
6
7 while i < 5:
8     # 2. 定义变量, 记录每一圈做了多少个俯卧撑
9     j = 0  每次进入内层循环, 都需要重置计数
10    # 3. 操场跑圈
11    print('操场跑圈中.....')
12    # 4. 做俯卧撑
13    while j < 3:
14        print('做了一个俯卧撑')
15        j += 1
16    # 一圈完整了, 圈数加1
17    i += 1
18
```

for 循环遍历

- 1 `for` 变量 `in` 字符串:
- 2 代码
- 3 `for` 循环也称为 `for` 遍历, 会将字符串中的字符全部取到

```
1  ● for i in 'hello':
2      # i 一次循环是字符串中的一个字符
3      print(i)
4
5  # range(n) 会生成 [0, n) 的数据序列, 不包含n
6  ● for i in range(5): # 0 1 2 3 4
7      # print(i)
8      print('操场跑圈...')
9
10 # range(a, b) 会生成 [a, b) 的整数序列, 不包含b
11 for i in range(3, 7): # 3 4 5 6
12     print(i)
13
14 # range(a, b, step) 会生成[a, b) 的整数序列, 但是每个数字之间的间隔(步长)是step
15 for i in range(1, 10, 3): # 1 4 7
16     print(i)
17
18
```

循环打印直角三角形

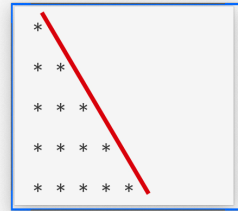
1	行	*个数
2	1	1
3	2	2
4	3	3
5	4	4
6	n	n


```

1  n = 5
2
3  # 2. 定义变量, 记录打印的行数
4  # j = 1 # 将要打印第一行
5  # while j <= n:
6  #     # 1. 定义变量记录一行打印的*个数
7  #     i = 1 # 将要打印第一个
8  #     while i <= j:
9  #         print('*', end=' ')
10 #         i += 1
11 #     print()
12 #     j += 1
13
14
15 # for循环打印三角形
16 for i in range(n):
17     for j in range(i+1): # i range(i) 不包含i
18         print('*', end=' ')
19     print()
20

```

行	*个数
1	1
2	2
3	3
4	4
5	5
6	6



Break 和 continue

1. **break** 和 **continue** 是 python 两个关键字
2. **break** 和 **continue** 只能用在循环中
3. **break** 是终止循环的执行, 即循环代码遇到 **break**, 就不再循环了
4. **continue** 是结束本次循环, 继续下一次循环, 即本次循环剩下的代码不再执行, 但会进行下一次循环

```
1 # 有五个苹果
2 # 1. 吃了三个苹果之后, 吃饱了. 后续的苹果不吃了
3 # 2. 吃了三个苹果之后. 在吃第四个苹果, 发现了半条虫子, 这个苹果不吃了,
4
5 for i in range(1, 6):
6     if i == 4:
7         print('吃饱了, 不吃了')
8         break # 终止循环的执行
9     print(f'正在吃标号为 {i} 的苹果')
10
```

for i in range(1, 6) > if i == 4

Debug: 18-break x

Debugger Console

pydev debugger: process 5832 is connecting

Connected to pydev debugger (build 201.6668.115)

正在吃标号为 1 的苹果
正在吃标号为 2 的苹果
正在吃标号为 3 的苹果
吃饱了, 不吃了

```
1 # 有五个苹果
2 # 1. 吃了三个苹果之后, 吃饱了. 后续的苹果不吃了
3 # 2. 吃了三个苹果之后. 在吃第四个苹果, 发现了半条虫子, 这个苹果不吃了, 还要吃剩下的苹果
4
5 for i in range(1, 6):
6     if i == 4:
7         print('发现半条虫子, 这个苹果不吃了, 没吃饱, 继续吃剩下的')
8         continue # 会结束本次循环, 继续下一次循环
9
10 print(f'吃了编号为{i}的苹果')
```

for i in range(1, 6)

Run: 19-continue x

C:\Develo\python38\python.exe C:/Users/nl/Desktop/43/day02/19-continue.py

吃了编号为1的苹果
吃了编号为2的苹果
吃了编号为3的苹果
发现半条虫子, 这个苹果不吃了, 没吃饱, 继续吃剩下的
吃了编号为5的苹果

1. break 和 continue 是 python 两个关键字

2. break 和 continue 只能用在循环中

3. break 是终止循环的执行, 即循环代码遇到 break, 就不再循环了

4. continue 是结束本次循环, 继续下一次循环, 即本次循环剩下的代码不再执行, 但会进行下一次循环

循环 else 结构

```
1  for x in xx:
2      if xxx:
3          xx  # if 判断条件成立会执行
4      else:
5          xxx  # if 判断条件不成立, 会执行
6  else:
7      xxx  # for 循环代码运行结束, 但是不是被 break
           终止的时候会执行
8
9  需求:
10     有一个字符串 'hello python', 想要判断这个字符串中
       有没有包含 p 这个字符, 如果包含, 就输出, 包含 p,
       如果没有 p , 就输出, 不包含
```

```
1  my_str = 'hello python!'
2  # my_str = 'hello itcast!'
3
4  for i in my_str:
5      if i == 'p':
6          print('包含p这个字符')
7          # 已经判断出来包含了, 是否还需要继续判断
8          break
9      else:
10         print('不包含p这个字符')
```

```
1  for x in xx:
2      if xxx:
3          xx  # if 判断条件成立会执行
4      else:
5          xxx  # if 判断条件不成立, 会执行
6  else:
7      xxx  # for 循环代码运行结束, 但是不是被 break 终止的时候会执行
8
9  需求:
10     有一个字符串 'hello python', 想要判断这个字符串中有没有包含 p 这
       个字符, 如果包含, 就输出, 包含 p, 如果没有 p , 就输出, 不包含
```

for i in my_str -> if i == 'p'

Run: 20-循环和else

C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day02/20-循环和else.py

包含p这个字符

Process finished with exit code 0

总结补充

```
1  num = 76
2  使用代码的方法, 求出这个数字的个位数和十位数
3  个位数: num % 10
4  十位数: num // 10
5
6
7  判断 if elif else
8
9  if 判断条件:
10     pass # 占位, 空代码 让代码不报错
11  elif 判断条件:
12     pass
13  else:
14     pass
15
16
17  循环: 重复做一件事 while for
18  while 判断条件:
19     pass
20
```

```
21  for i in xxx:
22      pass
23
24  break 和 continue,
25
```